LabPilot: A Self-Verifying Syringe Pump for Automated Sample Preparation

Laboratory 4.0 Challenge, **Axis 1: Automated Sample Preparation**

Introduction : LabPilot runs a digital protocol on an open-source syringe pump and accepts a sample only when *three independent witnesses agree*: the commanded motion, the mass measured by a scale, and what a camera sees. Any disagreement stops the run with a named fault and is logged.

1 Q1. Bench task and why it matters

Task automated: preparing a batch of liquid samples, that is, dispensing a set volume of liquid into each of several labelled containers from a written protocol. It is the repetitive step that precedes assays, formulation tests and stability studies.

Why it matters. (i) *Precision:* a volume error on the first container is carried through the rest of the batch. (ii) *Reproducibility:* two operators rarely run a protocol identically, and irreproducible experiments are a widely reported problem [1]. (iii) *Health:* repetitive manual pipetting is associated with hand and shoulder complaints among laboratory technicians [2, 3]. (iv) *Traceability and independent checking:* manual steps are mostly recorded on paper after the fact, and nobody checks what was actually dispensed, whereas pharmaceutical data-integrity guidance expects records that are attributable, contemporaneous and accurate [4].

Positioning. Open-source syringe pumps [6, **?**] and camera-based quality control on liquid handlers [5] both exist, so we do not claim either as new. Most low-cost dispensers still run *blind*: they assume a command always produces the right result. Our contribution is a small, low-cost, protocol-driven workflow where commanded, weighed and observed states are compared at every step, so that the machine knows whether the physical operation really happened.

2 Q2. What we build and how it is used

LabPilot is a single vertical syringe driven by a stepper motor and lead screw, dispensing into a container that sits on a small load cell. A fixed camera with a backlight watches the syringe and the container. An Arduino Uno with a CNC Shield V3 drives the motor, reads the load cell and runs an LCD with buttons; a Raspberry Pi runs the protocol, image processing, decision logic and log, and shows the protocol, live camera view, status and results on its screen (touchscreen or monitor). Typical use:

1. The operator loads a protocol (CSV/JSON file or a form on the touchscreen); the screen shows the next step and target.
2. The operator places the requested container; the scale tares and the camera confirms it is present and in position.
3. The system dispenses, stops on the measured mass, checks that all witnesses agree, logs the result, and asks for the next container.

Two layers. The *Arduino* is the hardware layer: it drives the stepper, reads the load cell and end-stop, and runs the fast, slow and stop dosing loop locally, so the pump stops on a mass threshold even if the Raspberry Pi is busy. The *Raspberry Pi* is the intelligence layer: it reads the protocol, sends each target to the Arduino over USB serial, receives mass and plunger-step readings back, analyses the camera image with OpenCV, compares the three witnesses, and writes the log.

```
protocol_id,sample_id,target_mL,tolerance_mL
P-001,A,5.0,0.2
P-001,B,3.0,0.2
P-001,C,5.0,0.2
```

Targets and tolerances come from the protocol, and the controller refuses values above the syringe capacity.

3 Q3. Main parts and how they connect

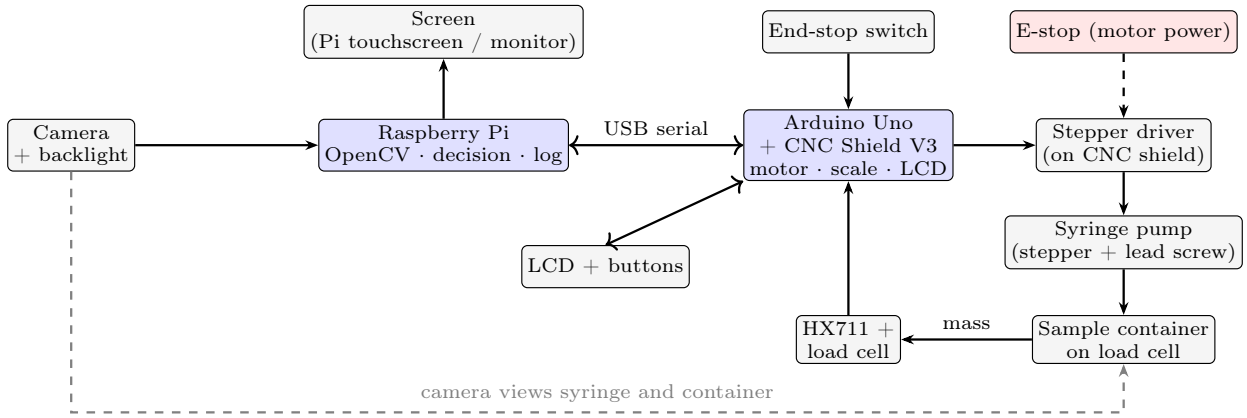


Figure 1. Block diagram. The Arduino moves the pump and reads the scale; the Raspberry Pi compares what the camera sees with what is expected and displays the result.

Part	Role
Mechanism	Open-source stepper syringe pump (30 mL syringe holds the whole demo without refill); 3D-printed and laser-cut frame, container holder, camera mount and spill tray.
Sensors	Load cell + HX711 (delivered mass, container presence); fixed camera with backlight (container position, plunger, liquid level, visible faults); end-stop switch (plunger limit).
Controller	Arduino Uno + CNC Shield V3 (stepper driver plugs into the shield) for motor, scale and LCD; Raspberry Pi for vision (OpenCV [8]), decisions, logging and display on its screen.
Safety	Physical emergency stop on motor power, pause button, spill tray, electronics kept away from liquids.

Table 1. Main parts.

4 Q4. What the minimum demo will show after 18 hours

Minimum demo:

- A protocol of 3 samples is loaded and executed automatically into 3 containers.
- **Pre-check:** dosing starts only if the camera sees the container in position and the scale registers it; otherwise nothing is dispensed.
- **Weight-driven dosing:** the pump runs fast, slows as the measured mass nears the target, and stops; the result is verified after the reading settles.
- **Live fault demonstration:** removing a container gives `NO_CONTAINER`; pinching the outlet tube gives `NO_FLOW` (plunger moves, no mass gain) or `MOTOR_STALL` (plunger blocked).
- One log record per sample plus a run summary, shown on screen, e.g.
  `P-001 | A | 5.0 mL | ACCEPTED  P-001 | B | - | NO_CONTAINER  P-001 | C | - | NO_FLOW`

Extra (only if the core is reliable): camera-based liquid-level estimate as a fourth cross-check, pause and resume when the container moves, a dilution step, timed mixing, a motorised container tray, a web dashboard.

Measured gain versus manual work (planned metrics). The same 10-sample batch is prepared by hand with a pipette and by LabPilot. We compare volume error and its spread (CV%), time per sample, errors caught before the sample leaves the bench, and completeness of the record. The scale itself is checked against a reference balance, following the gravimetric principle of ISO 8655-6 [7]. No results are claimed at this stage.

5 Q5. Monitoring, verification and logging

Before each operation the software knows what should happen (*expected state*). During and after it, the sensors report what did happen (*observed state*). Any mismatch aborts the sample, names the fault and is logged.

Check	Expected	Observed by	If not
Container	Present, in position, mass jump on placement	Camera + load cell	`NO_CONTAINER`, `MISPLACED`
Plunger	Moves as commanded, within end-stop limits	Camera + end-stop	`MOTOR_STALL`
Delivered mass	Rises in step with commanded volume	Load cell (tared)	`NO_FLOW`, `MISMATCH`, `OVERFILL`
Final result	Settled mass, converted to volume, within tolerance	Load cell (verdict)	REJECTED with deviation

Table 2. Expected versus observed state, and the fault raised when they differ.

Control and verification. Dosing is driven by the load cell (fast, then slow, then stop near target), and the verdict uses the mass after it has settled, since residual drops fall after the pump stops. For water, mass is converted to volume with its known density; other liquids would need their own calibration. The camera does not measure the volume. The load cell answers *how much was delivered*; the camera answers *what is physically happening*. Before dosing it checks that the container is present and in position (nothing is dispensed otherwise), and during dosing it checks that the plunger moves. Table 3 shows how the three witnesses are combined.

Motor command	Camera	Load cell	Decision
Push	Plunger moves	Mass matches command	Continue, then ACCEPTED
Push	Plunger moves	No mass gain	`NO_FLOW`
Push	Plunger still	No mass gain	`MOTOR_STALL`
Push	Plunger moves	Mass differs from command	`MISMATCH` / `OVERFILL`

Table 3. Agreement of the three witnesses (container checks come first, see Table 2).

Log. Every operation appends one timestamped record (append-only CSV/JSON on the Raspberry Pi), for example:

```
{protocol_id: P-001, sample_id: B, target_mL: 3.0, commanded_mL: 3.0,
weighed_mL: 3.1, camera: OK, tolerance_mL: 0.2, error: none,
status: ACCEPTED, t: 2026-10-16T09:41:07}
```

A run summary lists accepted, rejected and aborted samples and every detected error.

Feasibility and known limits. One controller, one actuator, one scale and one camera keep the 18-hour build realistic, and the build order protects the demo: pump and scale loop first, then the log, then the camera checks. Even if vision is unreliable, the load cell alone still meets the minimum demo. Limits: load-cell calibration, drift and settling time (target resolution about 0.1 g, to be characterised against a balance in Phase 2); camera sensitivity to lighting; the water-density assumption; and containers placed one at a time by the operator in the first version.

References

[1] M. Baker, "1,500 scientists lift the lid on reproducibility," *Nature*, vol. 533, no. 7604, pp. 452–454, 2016.

[2] M. G. Björksten, B. Almby, and E. S. Jansson, "Hand and shoulder ailments among laboratory technicians using modern plunger-operated pipettes," *Applied Ergonomics*, vol. 25, no. 2, pp. 88–94, 1994.

[3] G. C. David and P. Buckle, "A questionnaire survey of the ergonomic problems associated with pipettes and their usage with specific reference to work-related upper limb disorders," *Applied Ergonomics*, vol. 28, no. 4, pp. 257–262, 1997.

[4] U.S. Food and Drug Administration, "Data Integrity and Compliance With Drug CGMP: Questions and Answers, Guidance for Industry," Dec. 2018.

[5] S. U. Khan, V. K. Møller, R. J. N. Frandsen, and M. Mansourvar, "Real-time AI-driven quality control for laboratory automation: a novel computer vision solution for the Opentrons OT-2 liquid handling robot," *Applied Intelligence*, vol. 55, art. 524, 2025.

[6] B. Wijnen, E. J. Hunt, G. C. Anzalone, and J. M. Pearce, "Open-source syringe pump library," *PLoS ONE*, vol. 9, no. 9, art. e107216, 2014.

[7] ISO 8655-6:2022, "Piston-operated volumetric apparatus — Part 6: Gravimetric methods for the determination of measurement error," 2022.

[8] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.